# SBOM Supply Chain Risk Scoring

A contextual ranking of which vulnerable libraries to fix first

**William A. Davis**

M.S. Computer Science, Cybersecurity Engineering | CS 690 Graduate Research Project | Advisor: Dr. Omar Ochoa | Fall 2026

# What I'll cover

1

## The problem

Why CVSS order is not fix-first order

2

## The scoring model

Five factors, one explainable score

3

## The tool

How it works and how I checked it

4

## The results

Three targets set in advance

5

## Conclusions

Limitations and future work

# Modern software is mostly other people's code

**645**

third-party libraries in one web application (OWASP Juice Shop)

**93**

known vulnerabilities, spread across 34 of those libraries

**2021**

Executive Order 14028 makes SBOMs a requirement for federal software suppliers

*Generating an SBOM is now routine. Deciding what to fix first is not.*

# CVSS says how bad a flaw could be, not what to fix first

Two vulnerabilities, both CVSS 9.8:

### Library A

- Being exploited in the wild (on CISA's KEV list)
- 94% chance of exploitation (EPSS)
- Used directly by the application

**94.9** my score

### Library B

- No known exploitation
- 0.2% chance of exploitation
- Buried four levels deep

**34.2** my score

*A CVSS-only list calls them equal. The contextual score separates them by about 61 points and says why.*

# Three research questions

RQ1

## Differentiation

Does the contextual ranking differ meaningfully from a CVSS-only ranking of the same libraries?

RQ2

## Justification

When a library rises into the top 10, is the cause a factor CVSS does not capture?

RQ3

## Stability

Does the ranking stay substantially the same when individual weights change?

*The model and all three targets were fixed in writing (D1) and approved before any scoring code was written.*

# Five factors, one score out of 100

**S** — **Severity** — 30%

CVSS score ÷ 10

**E** — **Exploit chance** — 25%

EPSS: probability of exploitation in the next 30 days

**X** — **Already exploited** — 20%

1 if on CISA's KEV list, 0.5 if exploit code is public

**D** — **Depth** — 15%

1 ÷ depth: direct = 1.0, depth 2 = 0.5, depth 4 = 0.25

**A** — **Age** — 10%

Days since release ÷ 5 years, capped at 1

### Pillow 9.4.0, worked by hand

| | | |
|---|---|---|
| Severity | 0.88 × 30 | **26.4** |
| EPSS | 1.00 × 25 | **25.0** |
| On KEV | 1 × 20 | **20.0** |
| Direct | 1.0 × 15 | **15.0** |
| Age | 0.77 × 10 | **7.7** |

**94.1** out of 100

# Four free, public data sources

**OSV**

**Which vulnerabilities affect this exact version?**

Queried by package URL, e.g. pkg:pypi/pillow@9.4.0

**NVD**

**How severe is it?**

CVSS base score from the National Vulnerability Database

**FIRST EPSS**

**How likely is exploitation?**

Daily probability of exploitation in the next 30 days

**CISA KEV**

**Is it exploited now?**

Catalog of vulnerabilities with confirmed active exploitation

**Reproducible by design.** Every API response is cached, every run records its analysis date and a SHA-256 hash of the SBOM, so any result can be rerun exactly, offline.

# How the tool works

1

## Read

`parser.py`

Reads the CycloneDX SBOM

2

## Map

`graph.py`

Rebuilds the dependency graph and finds each library's depth

3

## Look up

`enrich.py`

OSV, NVD, EPSS and KEV; every response cached

4

## Score

`score.py`

Ranks libraries and writes one-line explanations

5

## Evaluate

`evaluate.py`

Compares with CVSS-only and tests stability

**Python 3** standard library only

**34** automated tests, all passing

**3** commands: enrich, rank, evaluate

# Test projects: six applications, four ecosystems

| | Juice Shop | Gitea | PyGoat | WebGoat | Etherpad | Wiki.js |
|---|---|---|---|---|---|---|
| **Language** | JavaScript | Go | Python | Java | JavaScript | JavaScript |
| **Package manager** | npm | Go modules | pip | Maven | npm | npm |
| **Libraries** | 645 | 231 | 33 | 190 | 794 | 1216 |
| **Direct dependencies** | 63 | 110 | 16 | 37 | 40 | 154 |
| **Deepest level** | 7 | 3 | 3 | 8 | 8 | 10 |
| **Vulnerable libraries** | 34 | 26 | 14 | 8 | 86 | 142 |
| **Known vulnerabilities** | 93 | 105 | 145 | 56 | 216 | 406 |
| **On CISA KEV** | 0 | 1 | 1 | 1 | 0 | 0 |

*Juice Shop, PyGoat and WebGoat are OWASP training applications. Gitea, Etherpad and Wiki.js are production software; Etherpad and Wiki.js were added in November as larger projects.*

# Juice Shop v20.2.0: JavaScript (npm)

*OWASP's deliberately insecure online shop, used to teach web security. Node.js back end, Angular front end.*

**645**
libraries

**63**
direct dependencies

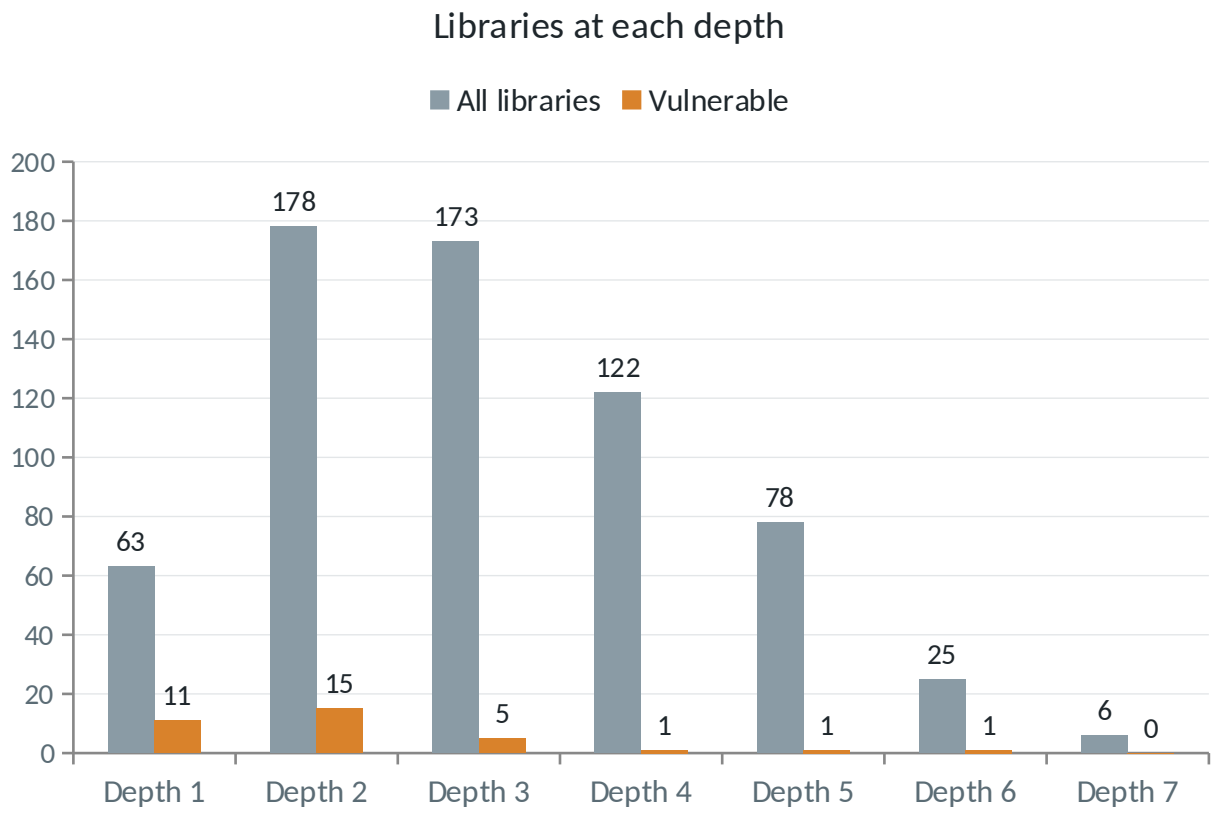
**7**
levels at the deepest

**34**
vulnerable libraries

**93**
known vulnerabilities

**0**
on CISA's KEV list

**Ranked #1:** jsonwebtoken 0.4.0 (66.5)



# Gitea v1.21.0: Go (Go modules)

*A self-hosted Git service, similar to GitHub. Real production software, included as a check on the training apps.*

**231**
libraries

**110**
direct dependencies

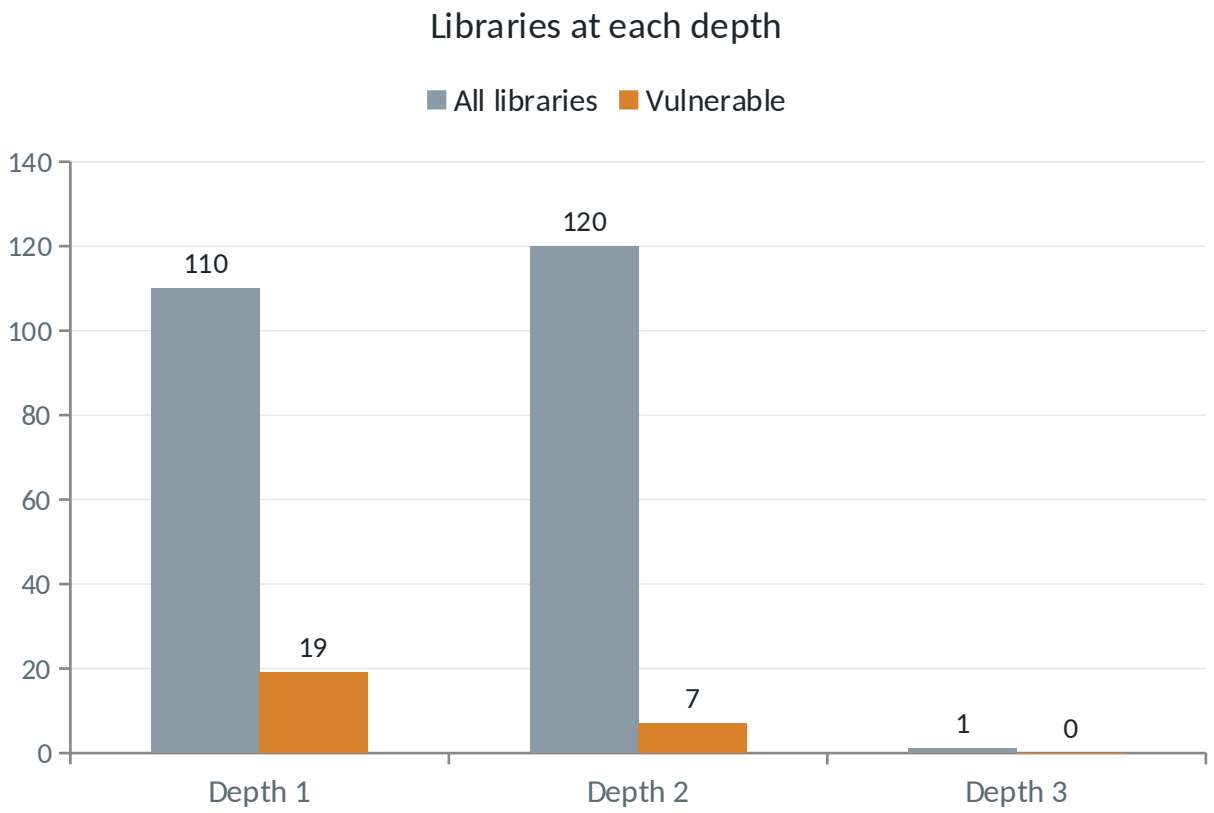
**3**
levels at the deepest

**26**
vulnerable libraries

**105**
known vulnerabilities

**1**
on CISA's KEV list

**Ranked #1:** brotli v1.0.5 (85.8)



# PyGoat v2.0.1: Python (pip)

*OWASP's deliberately vulnerable Django application for learning the OWASP Top 10.*

**33**
libraries

**16**
direct dependencies

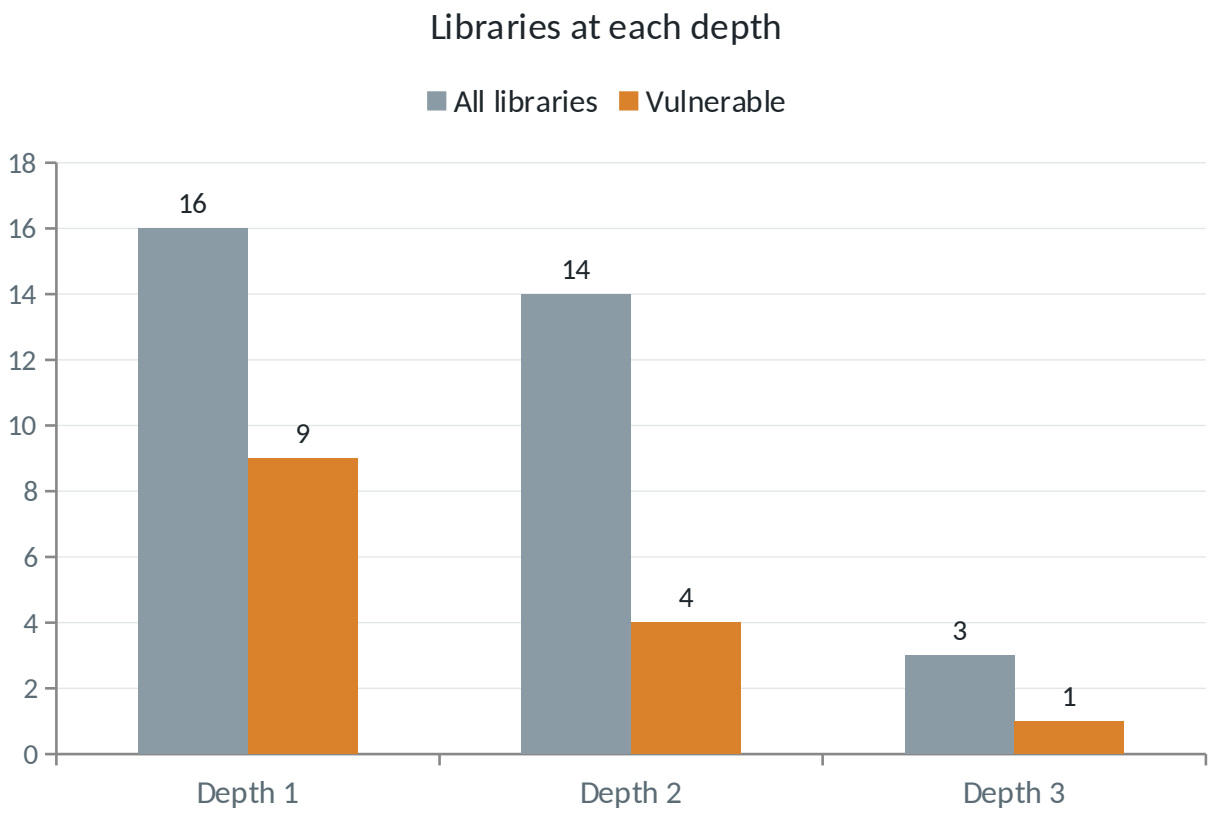
**3**
levels at the deepest

**14**
vulnerable libraries

**145**
known vulnerabilities

**1**
on CISA's KEV list

**Ranked #1:** pillow 9.4.0 (94.1)



# WebGoat v2026.4: Java (Maven)

*OWASP's deliberately insecure Spring Boot application for teaching application security.*

**190**
libraries

**37**
direct dependencies

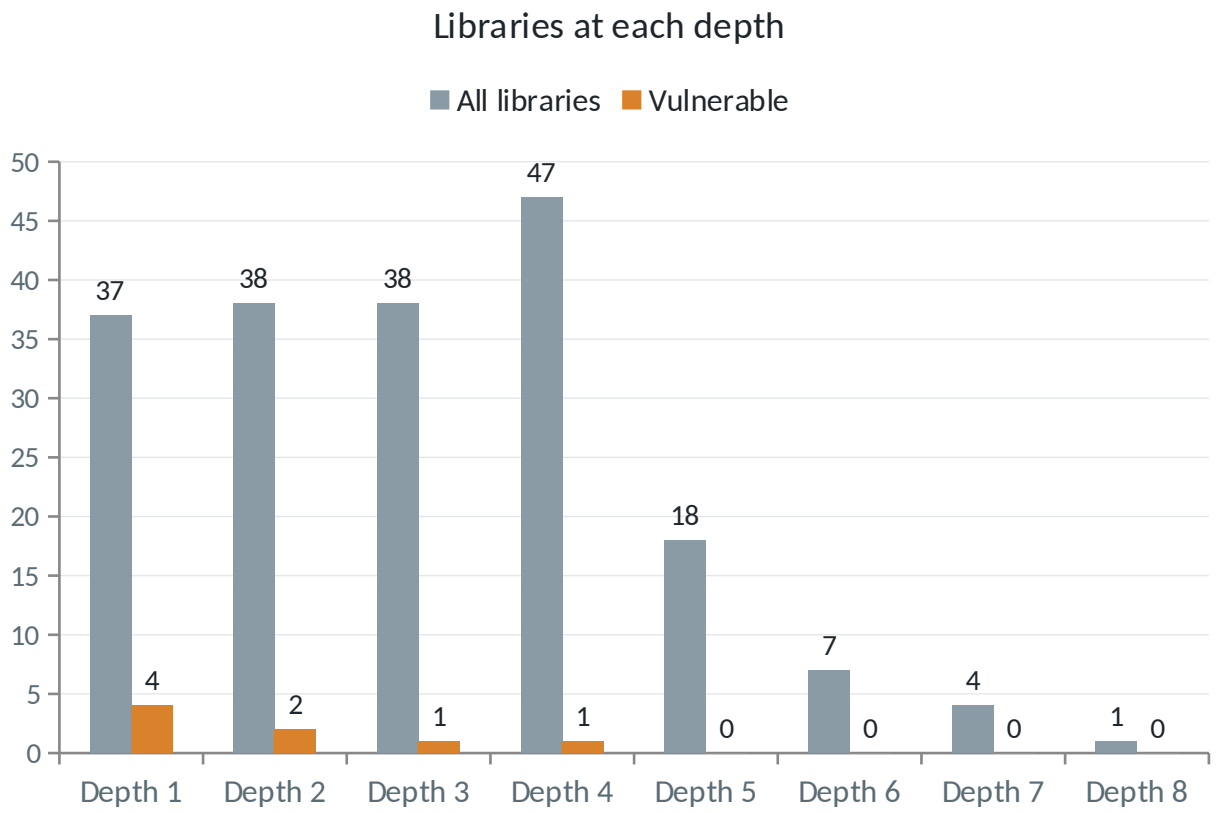
**8**
levels at the deepest

**8**
vulnerable libraries

**56**
known vulnerabilities

**1**
on CISA's KEV list

**Ranked #1:** xstream 1.4.5 (95.0)



# Etherpad 1.8.6: JavaScript (npm)

*A widely used open-source collaborative text editor. Real production software, added as a larger test project.*

**794**
libraries

**40**
direct dependencies

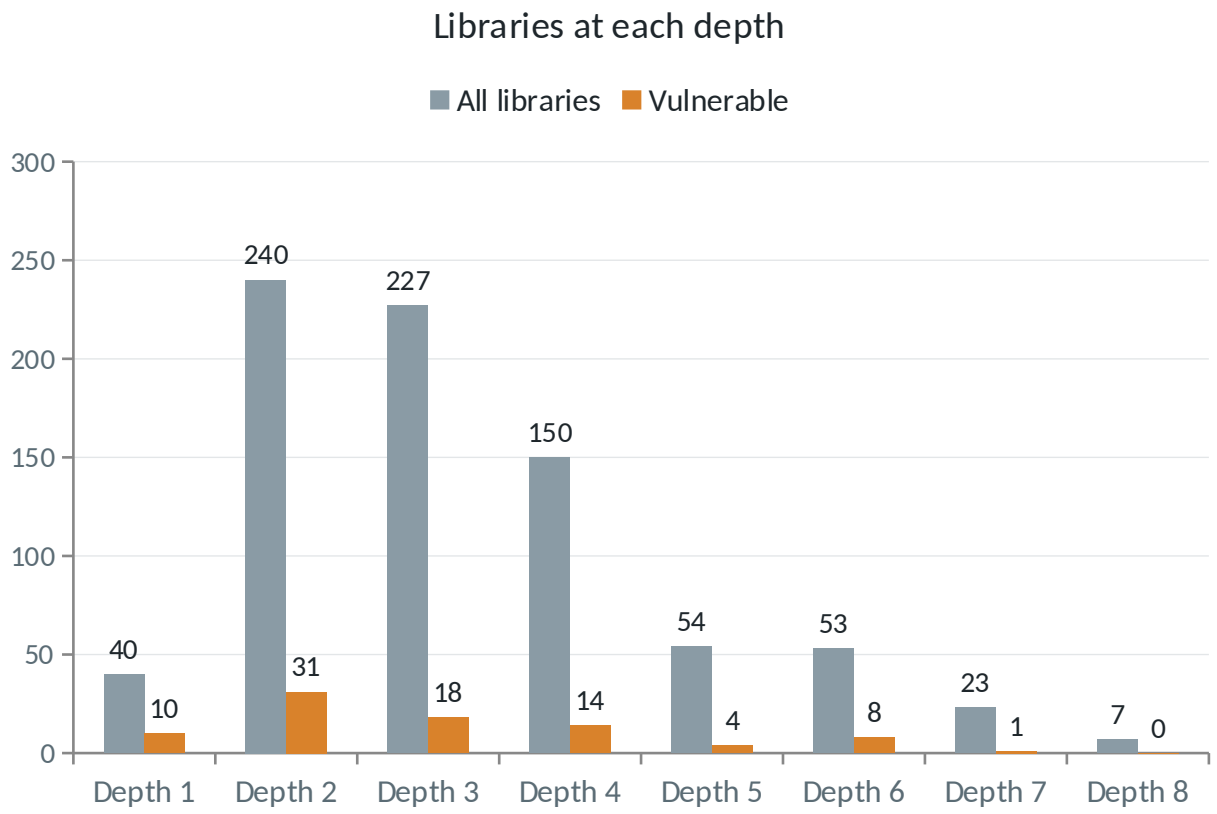
**8**
levels at the deepest

**86**
vulnerable libraries

**216**
known vulnerabilities

**0**
on CISA's KEV list

**Ranked #1:** ejs 2.6.1 (72.6)



# Wiki.js v2.5.300: JavaScript (npm)

*A popular open-source wiki platform that organizations install to run their own internal wiki. Real production software.*

**1216**
libraries

**154**
direct dependencies

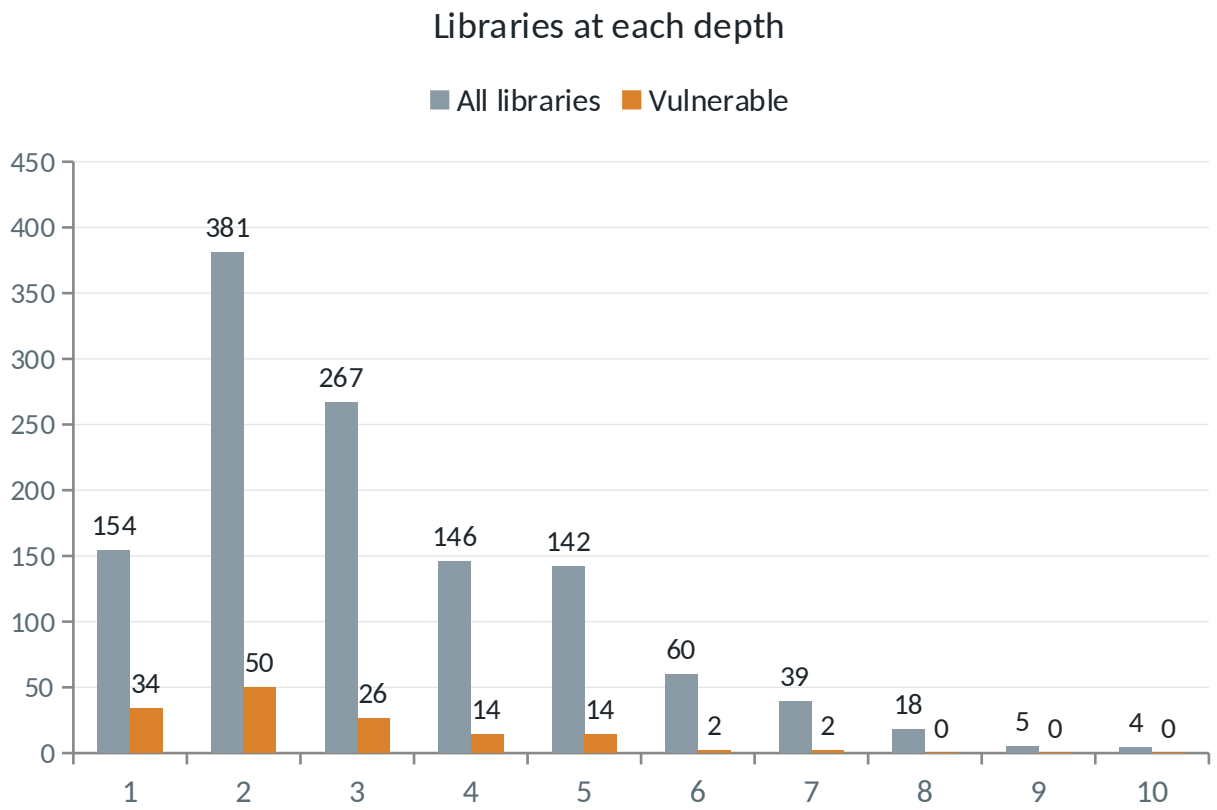
**10**
levels at the deepest

**142**
vulnerable libraries

**406**
known vulnerabilities

**0**
on CISA's KEV list

**Ranked #1:** y18n 4.0.0 (69.8)



# Is the tool correct? Two independent checks

## Dependency map

Compared with each package manager's own view

- **npm** `npm ls`
  Juice Shop 645/645, Etherpad 789/790, Wiki.js 1,216/1,216
- **Python** `pipdeptree`
  every dependency matched
- **Go** `go list`
  every dependency matched
- **Maven** `mvn dependency:tree`
  179 / 179 matched

## Vulnerability matching

Compared with the data behind the standard scanners

**141 / 141** PyGoat: every flaw in PyPI's data (used by pip-audit)

**77 / 77** Juice Shop: every advisory in npm's data (used by npm audit)

**+11** flaws in multer that npm audit misses because of how npm compares versions

# Example: PyGoat's top 10 versus CVSS alone

| My rank | Library | Score | CVSS-only rank | Why |
|---|---|---|---|---|
| **1** | **pillow 9.4.0** | **94.1** | **1** | **Exploited now (KEV), EPSS 1.00** |
| 2 | pyyaml 5.1 | 65.7 | 4 | CVSS 9.8, direct |
| 3 | django 4.1.7 | 64.6 | 2 | CVSS 9.1, direct |
| 4 | pyjwt 2.4.0 | 61.3 | 5 | CVSS 9.1, direct |
| 5 | cryptography 39.0.1 | 55.2 | 6 | CVSS 7.5, direct |
| **6** | **idna 3.4** | **48.6** | **11** | **Public exploit code** |
| 7 | werkzeug 2.1.2 | 47.4 | 10 | CVSS 7.5, direct |
| 8 | urllib3 1.26.9 | 46.6 | 9 | Public exploit code |
| **9** | **certifi 2022.12.7** | **44.9** | **3** | **CVSS 9.8, but EPSS 0.001, 2 deep** |
| 10 | sqlparse 0.3.1 | 43.7 | 7 | No exploitation evidence |

## What changed

- Pillow is #1 because it is being exploited right now.
- idna rises from #11 to #6: public exploit code.
- certifi falls from #3 to #9: CVSS 9.8, but almost no exploit activity and two levels deep.

# How I evaluated it: targets fixed in advance

**Baseline:** a CVSS-only ranking of the same libraries from the same data, so every difference comes from the other four factors alone.

RQ1

## Differentiation

At least 3 of the top 10 not in the CVSS-only top 10, in at least 3 projects; Kendall's τ also reported

RQ2

## Justification

Every library entering the top 10 has exploitation or depth as one of its two largest factors

RQ3

## Stability

Each weight moved ±0.05 and ±0.10 (20 runs per project): top-10 overlap ≥ 8 and τ ≥ 0.80 in 90% of runs

# Results: two targets met, stability met in 5 of 6 projects

## 3 projects

### Different top 10

At least 3 new libraries in the top 10: Etherpad (7), Wiki.js (8) and Juice Shop (5).

Met

## 23 of 23

### Justified

Every library that moved into a top 10 did so because it is exploited, likely to be, or used directly.

Met

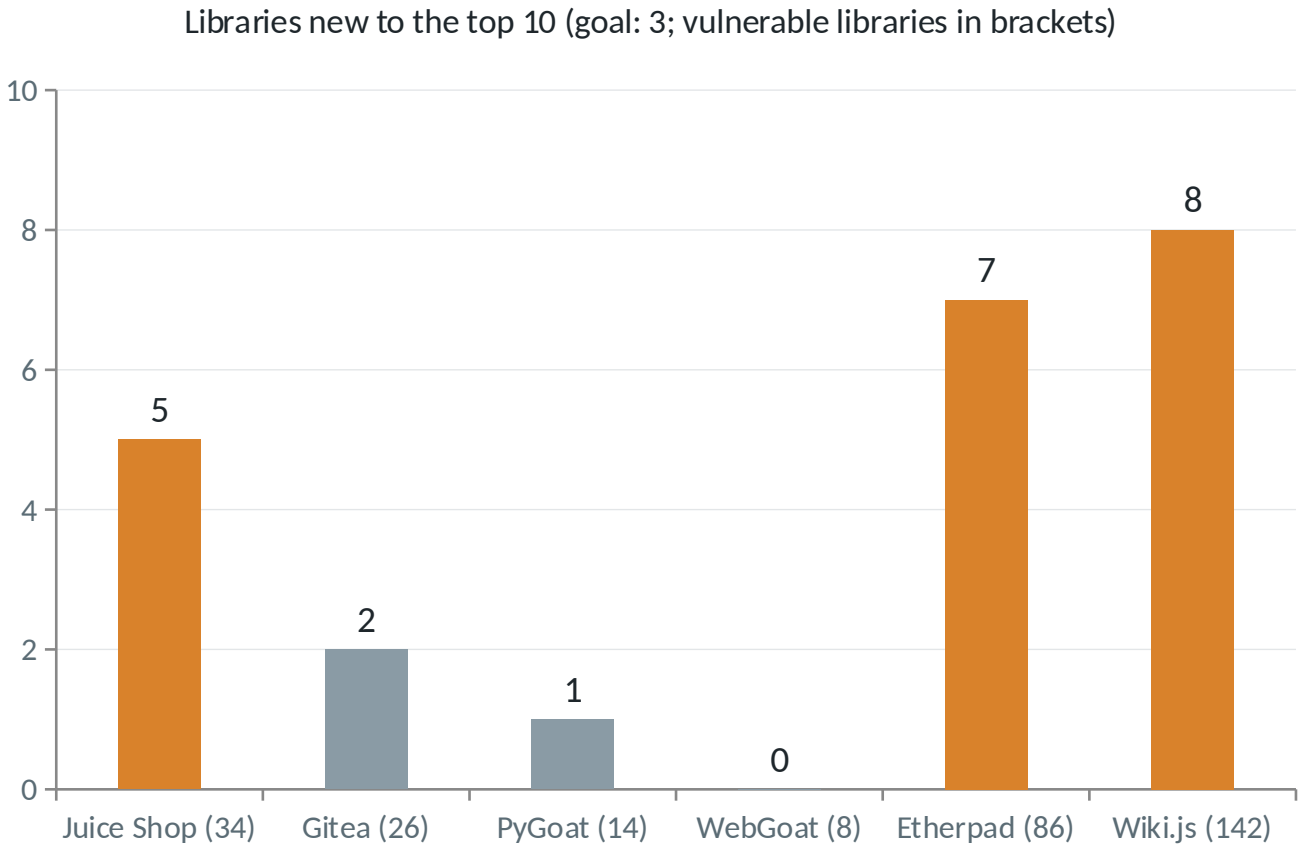
## 5 of 6

### Stable

107 of 120 weight changes held. Wiki.js held in 13 of 20, below the 18 required.

5 of 6 projects

# Large projects change the most




## What the chart shows

- All three large projects (34+ vulnerable libraries) meet the goal.
- WebGoat (8) and PyGoat (14) are too small: few libraries can ever enter a top 10.
- Etherpad and Wiki.js were added after the November review because the small projects could not fairly test the question.
- The weights were never changed.

# The order changed in every project

### Juice Shop

**31 of 34**

libraries changed position

**τ = 0.39**

### Gitea

**24 of 26**

libraries changed position

**τ = 0.68**

### PyGoat

**13 of 14**

libraries changed position

**τ = 0.58**

### WebGoat

**7 of 8**

libraries changed position

**τ = 0.64**

### Etherpad

**82 of 86**

libraries changed position

**τ = 0.40**

### Wiki.js

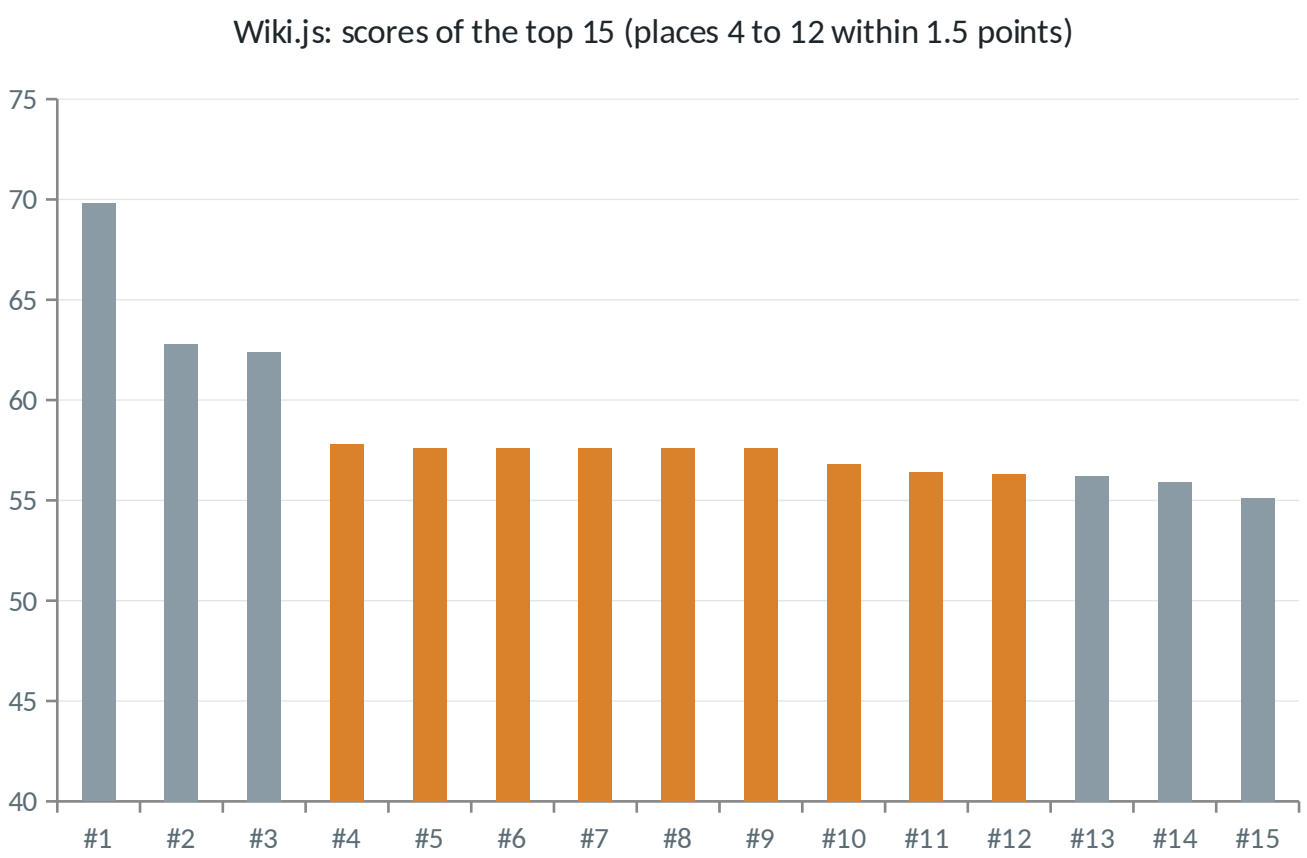
**138 of 142**

libraries changed position

**τ = 0.47**

**Kendall's τ** compares two rankings pair by pair: 1 means identical order, 0 means unrelated. Every project is well below 1, so the model really does reorder the list.

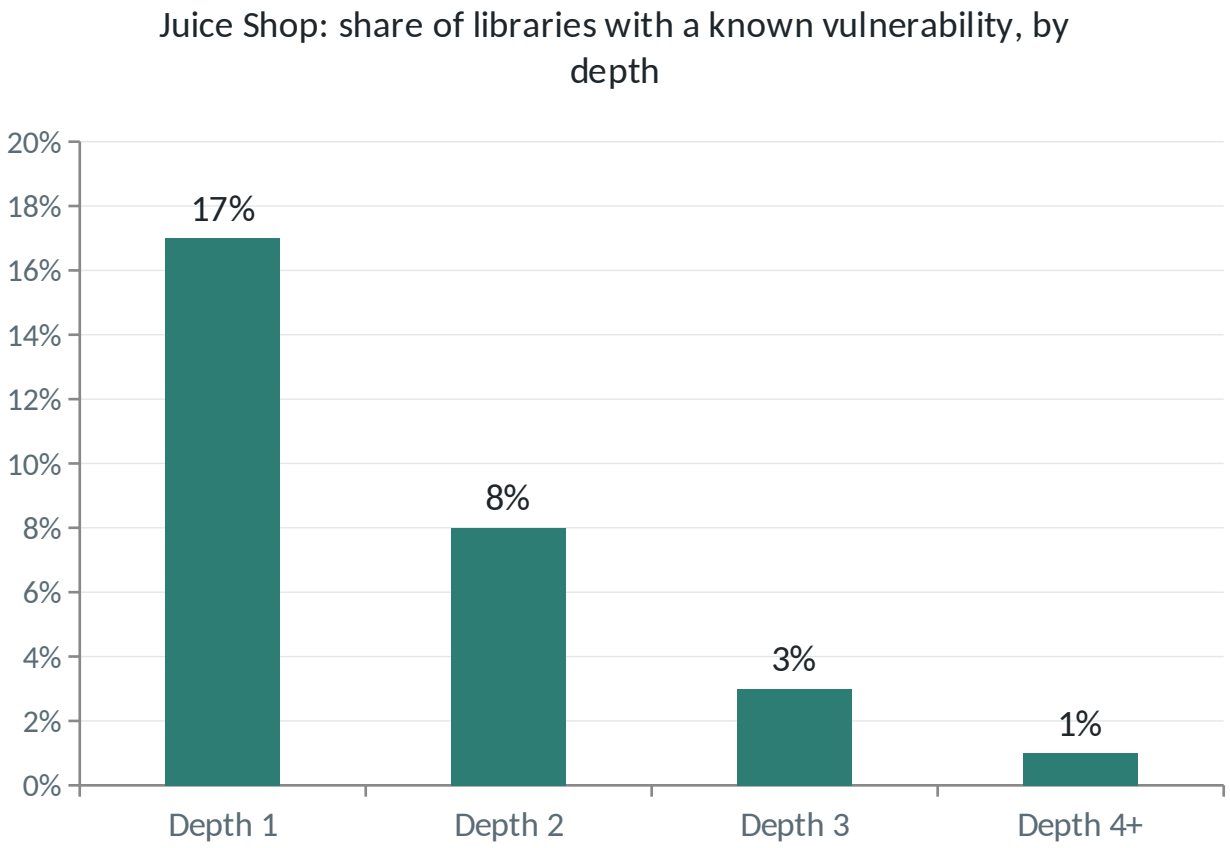
# Why Wiki.js held in only 13 of 20 runs




## What happened

- 13 libraries sit within 2 points of 10th place, about twice any other project.
- Failed runs lost top-10 overlap (6 or 7 of 10), mostly when the depth weight moved.
- The full ranking held: Kendall's τ stayed above 0.76 in every run.
- Reported as found, not removed or retuned.

# A supporting finding: risk sits near the top of the tree




## What it shows

- Direct dependencies are far more likely to carry a known vulnerability than deep ones.
- The same pattern holds in all six projects (Wiki.js: 22% of direct vs 8% at depth 4+).
- It supports giving depth a place in the score, and direct dependencies are the ones a team can upgrade itself.

# Limitations

### Depth is not reachability

It shows how close a library is, not whether the vulnerable code is ever called.

### EPSS covers only CVEs

Advisories without a CVE get 0 for exploit chance and are flagged.

### The data changes daily

Every run records its date and caches responses so it can be reproduced.

### No answer key

Nobody publishes the correct fix order, so I tested reasons and stability instead.

### Project size cuts both ways

Too few vulnerable libraries limits differentiation; very many near-ties make the top-10 cutoff fragile.

### Source data can over-report

OSV and PyPI sometimes list versions that are not really affected.

# Conclusions

1

## Context changes priorities

3 of 6 projects met the differentiation target, all three of the large ones, where a CVSS-only list is longest.

2

## Every change is explained

23 of 23 libraries that entered a top 10 did so for a reason CVSS cannot see, stated in one sentence.

3

## Robust to the weights

Held in 107 of 120 weight changes and in 5 of 6 projects; in Wiki.js only the top-10 cutoff moved.

4

## Practical and reproducible

Works from the SBOM alone with free public data, across four ecosystems, and reruns exactly.

*Contributions: a written scoring model, an open-source pipeline, validation across four ecosystems, and an evaluation against CVSS.*

# Future work

1

## Reachability

Check whether the app actually calls the vulnerable code, not just whether the library is present.

2

## Practitioner study

Ask security professionals whether the ranking matches the order they would fix things in.

3

## Broader testing

Large projects in more ecosystems, a tie-aware stability measure, and better Maven release dates.

# Thank you

CVSS tells you how bad a flaw could be. This tells you what to fix first, and why.

## Questions?

Code, data and reports: github.com/WILLIAMADAVIS47/Automated-Software-Supply-Chain-Risk-Scoring-Using-SBOM-Analysis